

Assist AI: The AI Chatbot Project

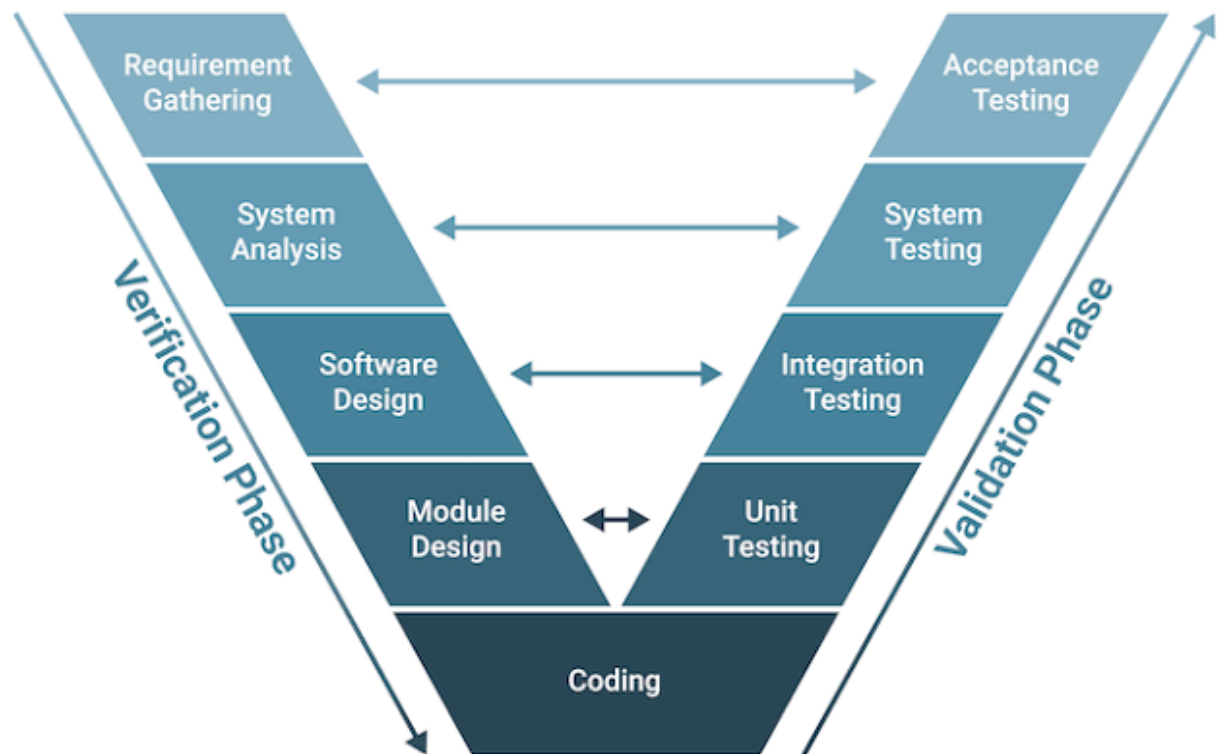
The Idea

Assist AI is a chatbot that answers customer questions on Whatsapp and takes appointments. Needs to be light and free. Asks for reviews; if review positive, redirect to google reviews page, else redirect to a troubleshooting form in Darija.

The Idea.....	1
Methodology.....	2
Customer Need.....	2
Requirement Gathering.....	3
The Existing Requirement Gathering Techniques:.....	3
The Techniques We Used:.....	3
Functional Requirements:.....	3
Non-Functional Requirements:.....	4
Constraints:.....	4
System Analysis.....	4
Actors.....	4
Data to Store.....	4
Use Cases.....	5
Business Flows.....	5
UML Diagram.....	5
System Boundaries.....	5
External Systems.....	6
Software Design.....	7
System Architecture.....	7
Architecture Diagram and State tree.....	8
UI Design.....	10
Module Design.....	11
ERD.....	13
Coding.....	13
The Technologies Used.....	13
Coding Standards.....	14
Unit Testing.....	14
Integration Testing.....	14
System Testing.....	14
Acceptance Testing.....	15
Time Planning.....	15

Milestones.....	16
Scope.....	17
What Will be Stored in the Database.....	17
What Will be Deleted periodically from the Database.....	17
References.....	17
Future directions.....	18
Concerns.....	18
To discuss.....	18

Methodology



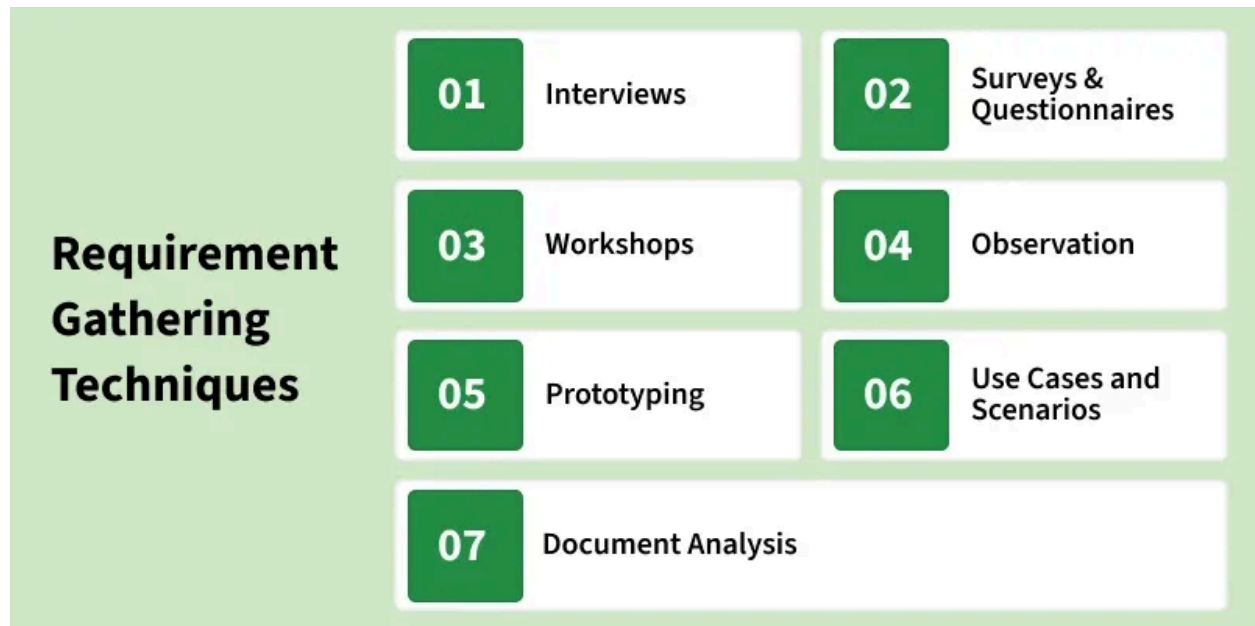
(Rashmi R. Athukorala, 2025)

Customer Need

As a company, there is a need for an AI chatbot that would answer customer questions about the company and its services. It would also need to set up appointments for the customers and take their feedback while providing a pleasant customer support experience.

Requirement Gathering

The Existing Requirement Gathering Techniques:



(GeeksforGeeks, 2025)

The Techniques We Used:

For this project's requirement gathering, we followed these two techniques:

Interview: We had an initial discussion with our supervisor to have an idea of what functionalities the tool should be providing as well as the involved parties. We also had a follow up discussion after outlining and analyzing those initial requirements to validate them.

Workshop/Brainstorming: After our initial discussion with our supervision we brainstormed the implications of each functionality to be able to outline the underlying requirements. We also brainstormed for what other requirements can support the functioning of the tool, and we researched what requirements similar tools followed.

Functional Requirements:

- Answer customer inquiries through WhatsApp.
- Support English, French, Arabic, and Darija.
- Schedule and manage appointments.
- Synchronize bookings with Google Calendar.
- Collect and classify customer reviews.

- Redirect positive reviews to Google Reviews and negative ones to a feedback form.
- Notify staff when human intervention is needed.
- Provide a dashboard with analytics and reports.
- Store customer, appointment, and conversation data.

Non-Functional Requirements:

- Fast response time.
- Secure data storage and user authentication.
- High system availability and reliability.
- User-friendly chatbot and dashboard.
- Scalable and modular architecture.
- Easy maintenance and future expansion.

Constraints:

- Must comply with WhatsApp's 24-hour messaging policy to use the WhatsApp Business API
- Depends on external AI and Google Calendar APIs.
- Limited by API quotas and free-tier usage.
- Must be completed within the project timeline and budget.
- Google calendar API needs careful availability checking to avoid conflict

System Analysis

Actors

- Customer
- Manager
- Administrator
- AI Chatbot

Data to Store

- Customers
- Appointments
- Reviews
- Feedback forms
- Knowledge base

Use Cases

- Ask question
- Book appointment
- Submit review
- Request human intervention
- View dashboard analytics

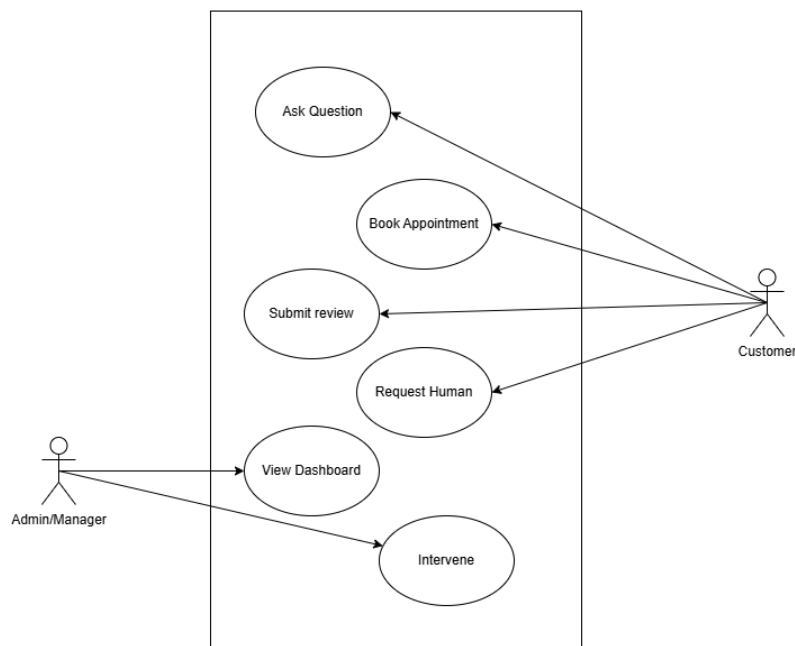
Business Flows

- Booking flow
- Q&A flow
- Human intervention flow

UML Diagram

<https://drive.google.com/file/d/1wSGjxNYvauSnM7nJpKqA50z1KUJwmuGe/view?usp=sharing>

AssistAI UML Diagram



System Boundaries

Customer Interaction

- Receive WhatsApp messages
- Detect message language
- Generate responses

- Handle FAQs and knowledge base queries

Appointment Management

- Check available slots
- Create temporary reservations
- Confirm appointments
- Cancel expired reservations

Feedback Management

- Request customer feedback
- Classify reviews as positive or negative
- Redirect users to review pages or troubleshooting forms

Human Need

- Detect requests for human assistance
- Notify managers
- Track escalation status

Dashboard

- User authentication
- Analytics visualization
- Review management
- Booking statistics
- Conversation statistics

Database

- Customers
- Appointments
- Reviews
- Feedback forms
- Knowledge base
- Analytics data

Security and Guardrails

- Input validation
- Prompt injection protection
- Content filtering

External Systems

WhatsApp Business API

- Message delivery
- Message reception

AI Providers

- Gemini
- Groq
- Qwen

Google Calendar API

- Calendar synchronization
- Availability verification

Tally Forms

- Feedback forms

Company Staff

- Human intervention decisions
- Manual customer support

Software Design

System Architecture

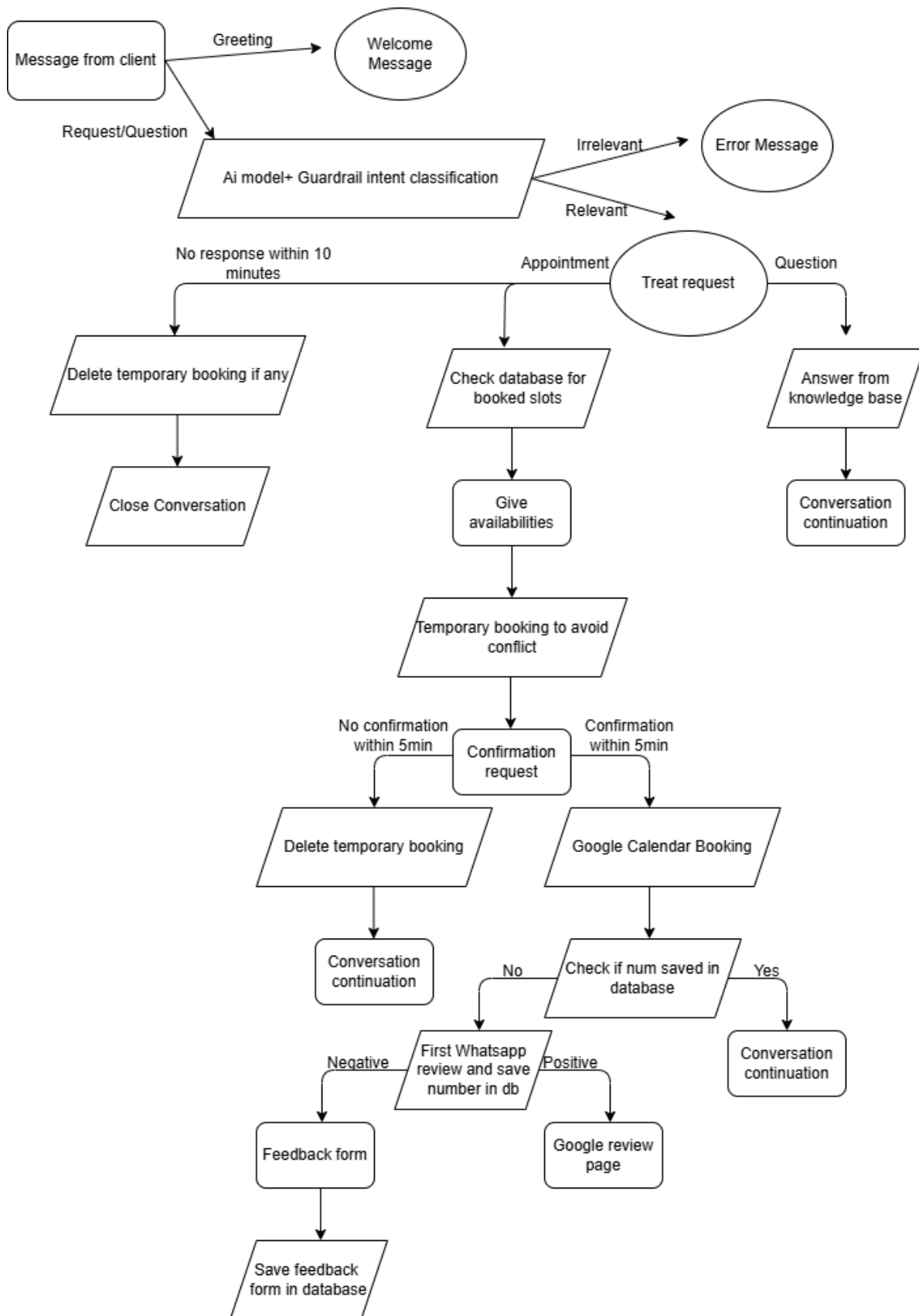
WhatsApp Interface Layer Receives and sends customer messages through WhatsApp Business API.	Chatbot Engine Responsible for: • Intent detection • Language detection • Knowledge retrieval • Response generation
Appointment Management Module Responsible for: • Availability checking • Temporary booking • Appointment confirmation • Calendar synchronization	Feedback Management Module Responsible for: • Review collection • Review classification • Form redirection
Dashboard Module Provides: • Review analytics • Booking analytics	Database Layer Stores: • Customers • Appointments

• Conversation analytics	• Reviews • Feedback forms • Knowledge base • Flagged requests
External Services • WhatsApp Business API • Google Calendar API • Gemini/Groq/OpenRouter • Tally Forms	

Architecture Diagram and State tree

<https://drive.google.com/file/d/14sPMfTEep6mKdTXBeJa3-c7i1R3eBaVq/view?usp=sharing>

AssistAI Architecture



UI Design

English, french, and arabic support

Log in page

Sign up page that requires company email

Home page:

- Total conversation number
- Total reviews number
- Total bookings
- 24 hour window reminder if applicable
- Human intervention alert

Chat Analytics:

- How many conversations per day/weeks/months
- AI chatbot usefulness
- Flagged requests
- Flagged responses
- Times human intervention was needed

Reviews Analytics:

- Positive reviews
- Negative reviews
- Visualization of reviews (pos/neg) over time
- Percentages of reviews
- Negative review list to read
- Categories of negative reviews causes

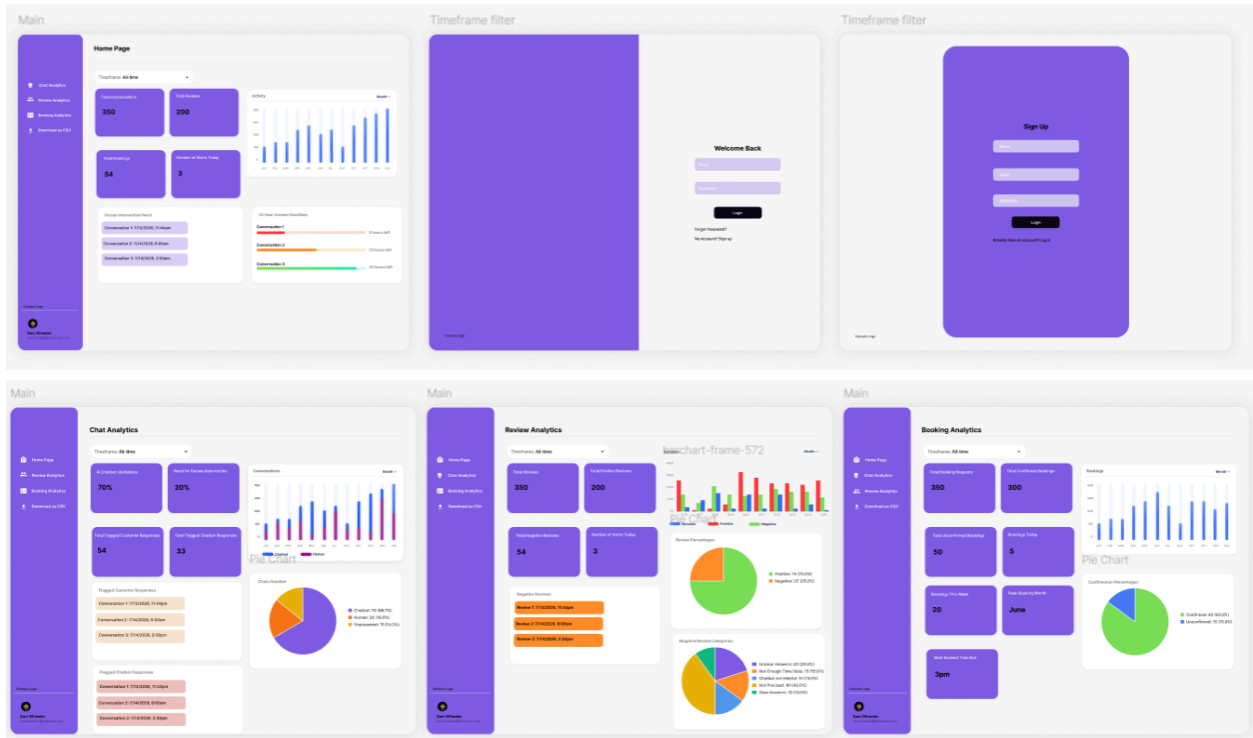
Booking Analytics:

- Total appointments
- Bookings today
- Bookings this week
- Booking trend (visualisation of appointments over days/weeks/months)
- Peak booking hours
- Peak booking days

Downloading data as csv

<https://www.figma.com/design/G1Zdxaj6V0AkpTTdPuyQzd/Analytics-Dashboard--Community-?node-id=2-3520&t=FMdM5sDS1pizd4CD-1>

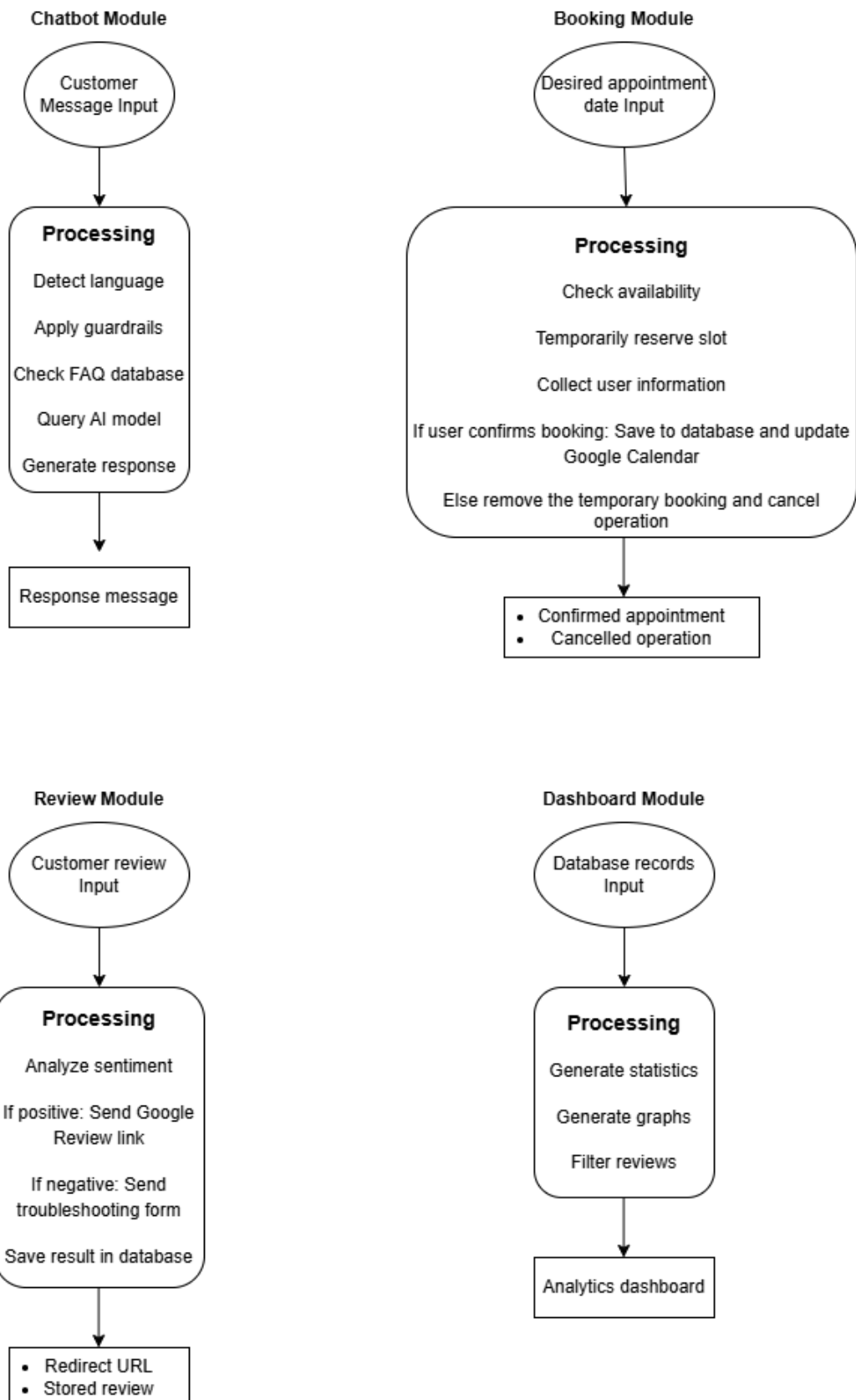
<https://www.figma.com/design/fDIrIsacdX9OpsBd8bydjLI/Analytics-Dashboard2?node-id=2-2522&t=NzlhOzV2QpiVO90B-1>



Module Design

https://drive.google.com/file/d/1TZVUTmDMxypELemb4BtNUnY_L2rrSVIX/view?usp=sharing

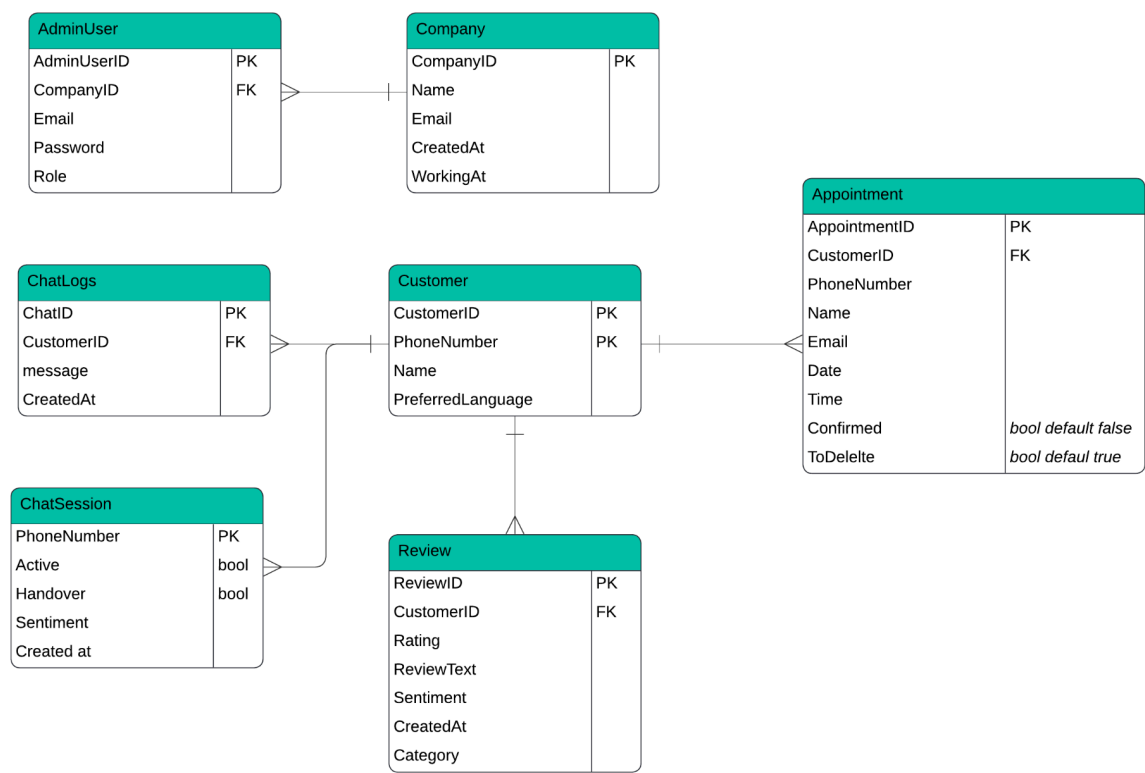
AssistAI Module Design



ERD

https://lucid.app/lucidchart/2af6613a-7849-4306-acbc-bac4e52fec4d/edit?viewport_loc=-1411%2C22%2C2340%2C1151%2C0_0&invitationId=inv_4e76ce1f-5807-4d5f-b2a8-a67587875e78

AssistAI ERD



Coding

The Technologies Used

Messaging • Bayleys API • WhatsApp Business API	Chatbot • https://github.com/codigoencaja/builderbot	AI • Gemini + Groq • Qwen • OpenRouter (Routes requests to different free models)
Backend • Node.js + Express	Database	Scheduling • Google Calendar API

• Python + FastAPI	• PostgreSQL Knex.js	
Guardrails https://github.com/guardrails-ai/guardrails https://github.com/NVIDIA-NeMo/Guardrails • Guardrail AI • Llama guard (already in groq)	Forms • Tally • Google Forms	Hosting • Local until deployed by client

Coding Standards

- Modular architecture
- REST API design
- Environment variables for secrets
- Error handling
- Logging
- Input validation

Unit Testing

Testing if the modules in this system are working.

The techniques we will use:

- Logic check
- Boundary checks
- Error handling
- Object-oriented checks

Integration Testing

Testing if the modules work together.

The approach we will follow is the bottom up integration testing.

System Testing

Testing the system as a whole and verifying both functional and non-functional requirements. A tool we can use is Postman. The types we can apply are:

- Functional Testing: Verifies that all features and functions work according to the specified requirements.
- Performance Testing: Evaluates the system's speed, responsiveness, and stability under different workloads.

- Load Testing: Checks how the application performs under expected user and transaction loads.
- Stress Testing: Determines the system's behavior when subjected to extreme workloads beyond normal limits.
- Security Testing: Identifies vulnerabilities and ensures data protection, authentication, and authorization mechanisms are effective.
- Usability Testing: Assesses the application's ease of use and overall user experience.
- Compatibility Testing: Verifies that the software works correctly across different devices, browsers, operating systems, and networks.
- Recovery Testing: Ensures the system can recover from failures such as crashes, hardware issues, or data loss.
- Reliability Testing: Measures the system's ability to perform consistently without failures over a specified period.

Acceptance Testing

Validates the software against business requirements and user needs and checks whether the system is ready for delivery and deployment.

- User Acceptance Testing (UAT): Performed by end users to verify that the software meets their requirements and works correctly in real-world scenarios.
- Business Acceptance Testing (BAT): Conducted to ensure that the software supports business processes and fulfills organizational goals and objectives.
- Contract Acceptance Testing (CAT): Verifies that the software complies with all requirements, terms, and conditions specified in the contract.
- Regulatory Acceptance Testing (RAT): Checks whether the software meets legal, regulatory, and industry standards before deployment.
- Operational Acceptance Testing (OAT): Validates the software's operational readiness, including security, backup, recovery, and maintenance procedures.
- Alpha Testing: Performed by developers or internal testers in a controlled environment to identify defects before releasing the software to external users.
- Beta Testing: Conducted by a limited group of real users in a real-world environment to gather feedback and identify any remaining issues.

Time Planning

<https://docs.google.com/spreadsheets/d/1yWdbqEwmRSouQ7vU1leWcmbI58i9U2630asCqMRKas8/edit?usp=sharing>

AssistAI Work Plan

PROJECT NAME	PROJECT MANAGER	DURATION	START DATE	END DATE
AssistAI	Soufiane Chakri	1 Month	07/13/2026	08/07/2026

	Phase 1	Phase 2	Phase 3	Phase 4
	Planning DURATION: 1 Week	Rapid Execution DURATION: 1 Week	Testing & Validation DURATION: 1 Week	Project Closure DURATION: 1 Week
	Define project scope, functional & non-functional requirements.	Initialize development workspace (Docker, Node.js).	Perform unit, integration, and system tests	Gather user feedback and supervisor sign-off.
	Map system boundaries	Code core multi-agent logic and API integrations.	Run load/stress tests on session concurrency	Compile final project documentation & setup guides.
	Develop project timeline and milestones.	Build database schemas and synchronization hooks.	Audit input sanitization and safety guardrails.	Prepare final internship report & presentation slides.
ACTIVITIES	Define functional requirements for booking, FAQs, and handover flows.	Set up Dockerized Node.js environment & PostgreSQL Knex migrations.	Write unit tests for input sanitization and logic gates.	Conduct User Acceptance Testing (UAT) sessions with supervisors.
	Map UML Use-Cases and System Boundaries.	Build the multi-agent routing service (Gemini, Groq, Qwan).	Execute integration tests between Express, Postgres, and LLM APIs.	Document setup guides, API endpoints, and disaster recovery steps.
	Design PostgreSQL schemas and AI Router (Gemini) state trees.	Code Google Calendar Sync, Tally webhook, and Supervisor Handover	Run performance testing on database locking mechanisms.	Finalize the V-Model validation report and internship defense slide deck.
OUTCOMES	Completed project scope document defining app functionalities.	Functional local backend with Baileys WhatsApp Gateway.	Comprehensive Unit, Integration, and System test reports.	Formal project handoff.
	Completed architectural diagrams and system boundary plans.	Database tracking active conversation states & session variables.	Validated prompt injection filters and payload safety limits.	Production-ready code repository with technical documentation.
	Drafted project schedule with milestones.	Fully functional booking synchronization and supervisor alert logic.	Optimizations applied for database lookup and API response latency.	Completed internship thesis report prepared for defense.

Milestones

Week 1:

Requirements Analysis and System Design Completed

- Functional and non-functional requirements approved
- UML use cases and system boundaries mapped
- Database schema and AI Router state tree designed

Week2:

Core System Implementation Completed

- WhatsApp gateway operational
- Multi-agent routing service implemented
- Booking synchronization and supervisor handover functional

Week3:

System Verification and Validation Completed

- Unit, integration, and system tests passed
- Performance benchmarks achieved
- Security and guardrail validation completed

Week4:

Project Acceptance and Handover Completed

- User acceptance testing completed
- Documentation finalized
- Final V-Model validation report submitted
- Project ready for demonstration and defense

Scope

[AssistAI_Scope.docx](#)

What Will be Stored in the Database

- Phone numbers
- Appointment dates and times
- Temporary slot booking
- Temporary client info for booking(name, phone number..)
- Temporary context table (names, dates, times, any specifications for context)
- Feedback forms
- Flagged items
- Knowledge base
- Keywords

What Will be Deleted periodically from the Database

- Temporary slot booking [When conversation is closed]
- Temporary client info for booking(name, phone number..) *
- Temporary context table (names, dates, times, any specifications for context)

References

Project	Stack	Architecture
---------	-------	--------------

gendonholaholo/Python-starter-kit-FastAPI-WhatsApp-AI-Chatbot	FastAPI + OpenAI + SQLAlchemy	Simple LLM + database
wassengerhq/whatsapp-chatgpt-bot-python	FastAPI + OpenAI + Wassenger	Modular service architecture
GonzaGomezDev/whatsapp-ai-assistant-starting-setup	FastAPI + LangGraph + PostgreSQL + Google Calendar	AI Agent with tools
Orchard-Run/orchard-meta-voice-agent	FastAPI + multiple LLM providers	Plugin architecture
boemer00/todo-work-agent	LangGraph + FastAPI + Google Calendar	Plan–Execute agent

Future directions

- Whatsapp reminder
- Ai extracting info from form
- Chatbot on different platforms (Meta)
- Voice calling the ai

Concerns

//How to scale an LLM api for free, limitations

//Stop customers from exploiting the ai>> Covered

//Prompt injection/exploitation prevention with online safeguards or ‘red flags’ documents (Lakera guard)

//How to know when to give feedback request>>timer, keywords for intent, registering given answers

To discuss

//Gatekeeping bad reviews concern to discuss

//Feedback for appointment only

//Reminder for appointments

//No google form, we use website or Tally

//Need for google review api key

//Reviews not for the company itself, reviews are about the chatbot

//Chatbot reaching out after appointment attended to follow up, only then request feedback

//Human intervention part; if customer requires human, the ai sends a notification>> 2 cases:

Manager either accepts or declines. If they accept, the client receives a 3minute notice message, else they receive an email to reach out to the company.